Appendix B

AI-Assisted Variations, Testing, and Review

Puzzle Arcade now has three playable games. That makes it a good place to practice AI-assisted development safely. The codebase is small enough for humans and AIs to understand but structured enough that changes can be reviewed against clear boundaries.

The purpose of this appendix is not to learn how to ask AI to vibe code a complete finished game for you. That will not help you learn anything of value. The aim of this appendix is to use AI as a variation partner. You define the design. You define the constraints. You choose the change. AI can suggest ideas, draft focused edits, generate test cases, and explain alternatives, but you remain responsible for the model, the rules, and the behavior.

You will apply that workflow to three kinds of variation. Word Guess will get better hints. Number Logic will get difficulty levels. Grid Puzzle will get a rule change that affects how tiles are toggled. Each variation should stay close to the files it affects. A word-game hint should not rewrite the shared shell. A grid toggle rule should not change the text-input command bar.

You will then test the game logic separately from the UI. The most important rules in Puzzle Arcade are ordinary C# code, so they can be checked without clicking through the browser every time. This reinforces one of the main lessons of the Blazor game chapters: interactive applications are easier to build when the UI is a projection of a testable model.

By the end of this appendix, you will have extended Puzzle Arcade, reviewed the architecture, and seen how modern C# helps make game logic readable, testable, and safe to change.

In this appendix, we will cover the following topics:

- Using AI to generate safe game variations
- Adding hints, difficulty levels, and rule changes



Solution code for the `PuzzleArcade` project up to the end of this appendix is found in the GitHub repository at the following link:

`https://github.com/markjprice/c-cpp-cs/tree/main/code/AppendixB`



Warning! To get the most out of many of the prompts in this appendix, I recommend loading the relevant chapters from the book's PDF, along with this appendix, into your AI so that it can refer to the material while providing more explanations. For example, Gemini Notebooks is good for this type of AI interaction because it makes it easy to load files to provide context for your questions.



Suggested chatbot prompts with example responses for this appendix can be found at the following link: `https://github.com/markjprice/c-cpp-cs/blob/main/prompts/appendix-b.md`

Using AI to generate safe game variations

Puzzle Arcade now has enough structure to make AI assistance useful. The project has shared components, shared state and action types, separate game models, and rule classes that contain the game-specific logic. That structure gives an AI assistant something to work with.

This matters because AI tools are much better when the code has clear boundaries. If all the logic is mixed into one large `.razor` file, an AI assistant might change the UI while trying to change a rule, or rewrite a model when asked to improve a message. But if the project is organized by responsibility, you can ask for smaller, safer changes.

In this section, you will learn how to ask an AI assistant to generate game variations without giving up control of the design. We will not let your AI invent an entirely new application. We want to use AI as a creative assistant inside the structure you have already built.

Why game variations are a good AI task

Puzzle games are well suited to AI-assisted variation because many changes are small, local, and easy to review. You can ask for a new word list, a different hint rule, an extra difficulty setting, or an alternative starting board. These are useful changes, but they do not require the AI assistant to understand every part of the application.

For example, this is a risky prompt:

```
Make the Puzzle Arcade games better.
```

That prompt is too vague. The AI assistant does not know whether "better" means harder, prettier, faster, more accessible, more colorful, or more complicated. It might rewrite files that already work.

This is a safer prompt:

```
In the PuzzleArcade Blazor project, update only Games/WordGuess/WordGuessRules.
cs.
Add a second word list for easier 4-letter words. Keep the existing GameState,
PuzzleAction,
GameStatusBar, and GameCommandBar types unchanged. Show the code changes and
explain them.
```

This prompt gives the AI assistant a target file, a specific change, and boundaries. It says what to change and what not to change.

That is the main skill in this section: ask AI for controlled variations, not uncontrolled rewrites.



Good practice: A good AI prompt names the specific feature, target files, constraints, and files that should not change. Avoid vague requests that invite the AI assistant to redesign working code.



Prompt: Please explain why Puzzle Arcade is a good project for practicing safe AI-assisted development. Focus on shared components, game-specific rule classes, models, and clear boundaries. Include an explanation of the difference between using AI as a variation partner and using AI as an automatic programmer.

Start from a working baseline

Before asking AI to change a game, make sure the current version builds and runs. This is especially important in a book project because each reader needs a reliable path through the code.

Run the app:

```
dotnet run
```

Open each game and do a quick manual check:

- **Word Guess**: Start the game, submit a valid guess, submit an invalid guess, reset the game.
- **Number Logic**: Start the game, submit a valid digit guess, submit an invalid digit guess, reset the game.
- **Grid Puzzle**: Start the game, click a tile, verify that neighboring tiles toggle, and reset the game.

You do not need a long test plan at this moment. You only need to know that the current project works before introducing AI-generated changes. If something breaks afterward, you know the break was introduced by the change.

This is the programming equivalent of saving your game before entering the boss fight.



Good practice: Before introducing an AI-generated change, make sure the project builds and the affected games still work. If the project breaks afterward, you know the change introduced the problem.

Give AI the project shape

An AI assistant does not automatically know the design intention behind your project. It sees code, but it might not understand which parts are deliberately shared and which parts are game-specific. Your prompt should explain the structure briefly.

Here is a useful setup prompt:

```
I am building a Blazor WebAssembly app named Puzzle Arcade.

The shared types are:
- GameState: a C# union with NotStarted, Playing, Won, and Lost.
- PuzzleAction: a C# union with StartGame, MakeGuess, SelectCell, ResetGame,
and ShowHint.
- GameStatusBar and GameCommandBar are shared Razor components.

Each game has:
- A model record that stores its state.
- A static rules class with an Update method.
- A Razor page that displays the model and dispatches PuzzleAction values.

Please suggest small variations that preserve this architecture. Do not suggest
replacing the shared model/action pattern.
```

This prompt teaches the AI assistant how to behave. It tells it that the architecture is not up for redesign. That is important because AI assistants often try to be helpful by changing more than you asked.

A good AI-assisted workflow is not "AI writes the program." It is "the developer defines the design, and AI helps create controlled changes inside that design."



Prompt: Give me a checklist for turning a vague AI coding request into a focused prompt with target files, constraints, and review steps.

Ask for ideas before asking for code

One safe way to use AI is to ask for variation ideas first, without asking for code. This lets you review the design before touching the project.

For example:

```
Suggest ten small variations for the Word Guess game in Puzzle Arcade.
Each variation must be beginner-friendly, must fit into the existing
WordGuessModel and WordGuessRules structure, and must not require a database or
JavaScript. For each idea, say which file would probably need to change.
```

A useful answer might suggest ideas like the ones shown in *Table Appendix B.1*:

Variation	Likely file
Add easy and hard word lists.	`WordGuessRules.cs`
Reject guesses that are not in an allowed word list.	`WordGuessRules.cs`
Add a first-letter hint.	`WordGuessRules.cs`
Show the alphabet with used letters marked.	`WordGuessPage.razor`
Change the number of allowed guesses.	`WordGuessModel.cs` and `WordGuessRules.cs`

Table Appendix B.1: Variations and their files

At this stage, you are reviewing ideas, not accepting code. That is safer. You can reject variations that are too large, too distracting, or not useful for readers.

The same approach works for the other games:

```
Suggest ten small variations for the Number Logic game. Keep the existing
NumberLogicModel, NumberLogicRules, and shared PuzzleAction pattern. Focus on
variations that change the rules or clues without changing the shared Blazor
components.
```

And:

```
Suggest ten small variations for the Grid Puzzle game. Keep the existing
GridPuzzleModel, GridBoard, GridPosition, and GridPuzzleRules design. Focus
on variations that can be implemented by changing board setup, tile state, or
toggle behavior.
```

These prompts are intentionally narrow. They ask for variations that fit the current project rather than variations that require a new architecture.



Good practice: Variation ideas are cheap to review. Code changes are more expensive to review. Ask AI for possible changes first, choose one, then ask for a focused implementation.



Prompt: Please explain why a developer should ask an AI assistant for variation ideas before asking it for code. Then suggest ten small variations for Word Guess that preserve the existing `WordGuessModel`, `WordGuessRules`, `GameState`, `PuzzleAction`, `GameStatusBar`, and `GameCommandBar` design.

Ask for one change at a time

After choosing a variation, ask the AI assistant to implement one change at a time.

Avoid this:

```
Add difficulty levels, hints, animations, score tracking, a new layout, and
better styling.
```

That is too much. If the generated code fails, it will be harder to know which change caused the problem.

Prefer this:

```
Add difficulty levels to Number Logic.

Requirements:
- Easy uses 3 unique digits and 10 guesses.
- Normal uses 4 unique digits and 8 guesses.
- Hard uses 5 unique digits and 7 guesses.
- Keep GameState and PuzzleAction unchanged.
- Keep GameCommandBar unchanged.
- Update only the NumberLogic model, rules, and page if needed.
- Show the changed files only.
```

This prompt gives the AI assistant enough detail to produce a focused change. It also protects shared code from unnecessary edits.

The more specific the request, the easier the result is to review.



Good practice: When the project already has a working structure, ask AI to show only the files that need to change. A large patch with unrelated edits is harder to review and easier to get wrong.

Ask AI to explain the change

When AI produces code, do not paste it blindly into the project. Ask it to explain the change in terms of the existing architecture.

For example:

```
Explain how this change fits the Puzzle Arcade pattern:
State + Action -> Rules -> New State -> UI.
Also list any files that should not have changed.
```

A good explanation should say something like this:

- The page still dispatches `PuzzleAction` values.
- The rule class still owns the game rules.
- The model still stores the current game state.
- The shared status and command components are unchanged.

If the explanation says that the AI changed the shared command bar to support one game-specific feature, that might be a warning sign. Shared components should only change when the shared behavior changes.



Good practice: Generated code that builds can still be a bad change. If you cannot explain how it fits the model, rules, actions, and UI flow, do not keep it yet.

Ask AI to produce review checkpoints

AI can also help you review its own output, but you should still make the final judgment. Ask for a checklist that you can verify manually.

For example:

```
Give me a manual review checklist for this change. Include build checks,
gameplay checks, and code-structure checks.
```

A useful checklist for a game variation might include those shown in *Table Appendix B.2*:

Check	Question
Build	Does `dotnet build` succeed?
Start	Can the game still start from `NotStarted`?
Play	Does the main action still work while `Playing`?
End	Does the game still move to `Won` or `Lost` correctly?
Reset	Does `ResetGame` return to a clean initial model?
Boundaries	Did the change avoid unrelated shared components?
Readability	Is the new rule in the correct rule class?

Table Appendix B.2: Checklist for a game variation

This is a good habit even when you write the code yourself. AI makes it more important because generated code can look plausible while still violating the project's design.

Use AI for data generation

One of the safest AI tasks is generating data rather than changing logic.

For Word Guess, AI can create word lists:

```
Generate a list of 50 common 5-letter English words suitable for a beginner
word puzzle. Use lowercase letters only. Avoid proper nouns, plurals ending
in s, obscure words, offensive words, and words with accents or punctuation.
Return the result as a C# string array initializer.
```

Even here, you must review the output. AI might include an obscure word, a repeated word, or a word with a meaning you do not want in a book that children might read. But reviewing a list is usually safer than reviewing a full rewrite of a rule class.

For Number Logic, AI can generate sample test cases:

```
Generate 12 test cases for the Number Logic CompareGuess method. Each test case
should include a 4-digit secret with unique digits, a 4-digit guess with unique
digits, the expected exact match count, and the expected misplaced match count.
Return them in a table.
```

For Grid Puzzle, AI can generate starting boards:

```
Generate ten 3x3 starting boards for a Lights Out-style puzzle. Use strings
where 1 means On and 0 means Off, such as "101". Avoid the all-off board.
Return each board as a call to GridBoard.FromRows.
```

Generated data still needs review. A generated grid might be too easy, too hard, or unsolvable depending on the rule set. But this is still a good use of AI because the requested output is constrained and easy to inspect.



Good practice: Word lists, test cases, and starting boards are good AI tasks because they are constrained and easy to inspect. Still review the output for duplicates, unsuitable words, invalid cases, and poor gameplay.



Prompt: Please explain why generated word lists, board layouts, and test cases are often safer AI tasks than asking AI to rewrite a rule class.

Use AI for small UI variations

AI can also help with small UI improvements. The safest prompts are specific about which files may change.

For example:

```
Improve the visual styling of the Grid Puzzle board. Change only wwwroot/css/
app.css. Do not change any C# or Razor files. Keep the existing CSS class
names. Make the on and off states easier to distinguish.
```

This kind of prompt is safer because it limits the change to CSS. If the game logic changes, you know the AI did not follow the instruction.

You can also ask for accessibility improvements:

```
Review the Grid Puzzle page for accessibility improvements. Do not change
the game rules. Suggest changes that make the tile buttons clearer to screen
readers and keyboard users. Show the Razor changes separately from the CSS
changes.
```

Accessibility suggestions are worth taking seriously because puzzle games can easily become too visual. A tile that only changes color might not be clear to all players. Text labels, button names, focus styles, and instructions can make the game easier to use.

Use AI for rule variations

Rule variations are more interesting, but they require tighter prompts.

For Word Guess:

```
Add a ShowHint variation to WordGuessRules.

Requirements:
- The first hint reveals the first letter.
- The second hint reveals the last letter.
- Later hints show "No more hints available."
- Hints must not consume a guess.
- Keep GameCommandBar unchanged.
- Keep the existing GuessResult union unchanged.
- Show only the files that need to change.
```

For Number Logic:

```
Add a harder mode to Number Logic where the secret has 5 unique digits.

Requirements:
- Do not change GameState or PuzzleAction.
- Preserve the existing CompareGuess behavior.
- Update validation so the required length comes from the model.
- Make sure the UI maxlength matches the current secret length.
- Show the changed code and explain why each file changed.
```

For Grid Puzzle:

```
Add a variation to Grid Puzzle where the toggle pattern includes diagonal
neighbors.

Requirements:
- Keep the page unchanged.
```

```
- Prefer changing GridPosition or GridBoard rather than GridPuzzlePage.
- Do not change GameState or PuzzleAction.
- Explain how the change affects the existing board behavior.
```

Notice how each prompt protects the architecture. You are not asking AI to “add a feature” in the abstract. You are telling it where the feature belongs.

Keep prompts close to the code

Good prompts name the exact types and files that matter.

Weak prompt:

```
Make the hints better.
```

Better prompt:

```
In Games/GridPuzzle/GridPuzzleRules.cs, improve the ShowHint behavior. The hint
should depend on the current board state, but it should not solve the puzzle.
Do not change GridPuzzlePage.razor or the shared components.
```

Even better prompt:

```
In Games/GridPuzzle/GridPuzzleRules.cs, update the ShowHint case. If the board
has more than five tiles on, show "Try reducing the number of on tiles." If the
board has one to five tiles on, show "Look for a move that clears a cluster."
If the board is already solved, show "The puzzle is already solved." Do not
change GridPuzzlePage.razor, GridBoard.cs, GameState.cs, or PuzzleAction.cs.
```

The last prompt is easiest to verify because the expected behavior is explicit. AI assistants work best with clear requirements.

Keep the human responsible

AI-generated code can be useful, but it is still your responsibility. In a learning project, that responsibility is part of the point. You are not only trying to make the game work. You are trying to understand why it works.

Before accepting an AI-generated change, ask yourself the questions shown in *Table Appendix B.3*:

Question	Why it matters
Does the change build?	Broken code is not an improvement.
Does the change fit the existing pattern?	A useful feature should not damage the architecture.
Did the AI change unrelated files?	Unrelated edits increase risk.
Can I explain the change?	Code you cannot explain is hard to maintain.

Can I test the change manually?	A game variation should have observable behavior.
Did the change make the game clearer or merely more complicated?	More code is not automatically better gameplay.

Table Appendix B.3: Questions to ask before accepting AI-generated changes

The last question is especially important for games. It is easy to add rules. It is harder to add rules that make the game more fun. A beginner project should stay small enough that the player understands what is happening.

A safe AI-assisted workflow

Use this workflow when generating variations:

1. Make sure the current project builds and runs.
2. Ask AI for variation ideas, not code.
3. Choose one small variation.
4. Write a prompt that names the relevant files and boundaries.
5. Ask for changed files only.
6. Review the explanation before applying the code.
7. Build and run the project.
8. Manually test the affected game.
9. Reject changes that rewrite unrelated code.
10. Commit or save the working version before trying another variation.

This workflow slows you down a little, but it prevents the most common AI-assisted coding problem: accepting a large change that you cannot easily review.

Small, controlled changes are easier to understand, easier to test, and easier to undo.

What AI should not do here

There are also tasks you should not hand over completely:

- Do not ask AI to replace the project architecture unless you are deliberately studying alternative designs.
- Do not ask AI to generate a large new game and paste it into the project without reading it.
- Do not accept changes to `GameState`, `PuzzleAction`, or shared components unless the change is genuinely shared across the arcade.
- Do not let AI remove type safety to make code shorter. For example, replacing the action union with strings would be a step backward.
- Do not accept a variation merely because it is clever. Clever code is not always good teaching code.

The standard for your AI-generated games should not be “Could this be done?” The standard should be “Can I or another beginner understand it, run it, and learn a useful C# idea from it?”

Using AI as a variation partner

The safest and most productive role for AI in this appendix is a variation partner. You define the structure. You define the boundaries. You decide which ideas are worth keeping. AI helps generate possibilities, draft small changes, suggest test cases, and explain alternatives.

That is a healthier relationship with coding assistants than treating them as automatic programmers. The developer remains responsible for the model, the rules, and the final behavior.

Puzzle Arcade is a good project for practicing that discipline because each game is small enough to understand. If an AI suggestion breaks the architecture, you can see it. If a change belongs in a rule class but appears in a Razor page, you can spot that. If a prompt is too vague, the generated code will probably show you why.

The next section applies this approach to concrete variations: adding hints, difficulty levels, and rule changes. You will see how to extend the games while keeping the shared state, action, and component patterns intact.

Adding hints, difficulty levels, and rule changes

The previous section showed how to use AI safely by asking for small, controlled variations. In this section, you will apply that idea to Puzzle Arcade.

You will make three kinds of changes, as shown in *Table Appendix B.4*:

Game	Variation	Main idea
Word Guess	Better hints	The first hint reveals the first letter, and the second hint reveals the last letter.
Number Logic	Difficulty levels	Easy, normal, and hard modes change the length of the hidden sequence and the number of guesses.
Grid Puzzle	Rule change	A different toggle pattern changes how the board behaves.

Table Appendix B.4: Variations for the three puzzle games

These are deliberately small changes. They do not replace the architecture. They do not change `GameState`. They do not replace `PuzzleAction`. They do not rewrite the shared status or command components. Each change fits into the model, rules, and UI boundaries already built.

That is the safest way to use AI in a project like this. You start with working code. You choose one variation. You identify the files that should change. You reject suggestions that rewrite unrelated parts of the project.

Adding better hints to Word Guess

The current Word Guess hint only tells the player how many letters are in the hidden word. That is not very useful, because the player already learns the required length as soon as they enter a guess.

A better hint system can reveal a little information without giving away the answer. We will use these rules:

1. First hint: Reveal the first letter.
2. Second hint: Reveal the last letter.
3. Later hints: Tell the player that no more hints are available.
4. Guess count: Hints do not consume guesses.

This is a good AI-assisted change because it is small and local. Most of the change belongs in `WordGuessModel.cs` and `WordGuessRules.cs`.

A safe prompt for an AI assistant would be:

```
In the PuzzleArcade Blazor project, add a better hint system to Word Guess.

Requirements:
- The first hint reveals the first letter of the hidden word.
- The second hint reveals the last letter of the hidden word.
- Later hints show "No more hints available."
- Hints must not consume guesses.
- Do not change GameState, PuzzleAction, GameStatusBar, or GameCommandBar.
- Prefer changing only WordGuessModel.cs and WordGuessRules.cs.
- Show the changed files only.
```

Implementing improved Word Guess game hints

Now make the change yourself so you understand what the AI assistant should have done:

1. In `Games/WordGuess/WordGuessModel.cs`, add a `HintsUsed` property to the model, as highlighted in the following code:

```
namespace PuzzleArcade.Games.WordGuess;

using PuzzleArcade.Games.Shared;

public sealed record class WordGuessModel(
    GameState State,
    string Answer,
    IReadOnlyList<GuessAttempt> Attempts,
    int MaxGuesses,
    int HintsUsed,
    string Message)
```

```
{
    public int AttemptsUsed => Attempts.Count;

    public int AttemptsRemaining =>
        Math.Max(0, MaxGuesses - AttemptsUsed);

    public bool CanGuess => State is Playing && AttemptsRemaining > 0;
}
```

2. In `Games/WordGuess/WordGuessRules.cs`, the `CreateInitial` method needs to pass `HintsUsed: 0`, as shown highlighted in the following code:

```
public static WordGuessModel CreateInitial() =>
    new(
        State: new NotStarted(),
        Answer: "",
        Attempts: [],
        MaxGuesses: 6,
        HintsUsed: 0,
        Message: "Start the game to choose a hidden word.");
```

3. The `StartNewGame` method also needs to reset the hint count:

```
private static WordGuessModel StartNewGame(int maxGuesses)
{
    string answer = Words[Random.Shared.Next(Words.Length)];

    return new(
        State: new Playing(TurnsTaken: 0),
        Answer: answer,
        Attempts: [],
        MaxGuesses: maxGuesses,
        HintsUsed: 0,
        Message: "A hidden word has been chosen. Make your first
guess.");
}
```

4. Add the following helper method to `WordGuessRules`:

```
private static WordGuessModel ShowWordHint(WordGuessModel model) =>
    model.HintsUsed switch
    {
        0 => model with
        {
            HintsUsed = 1,
            Message = $"Hint 1: the word starts with {char.
```

```
ToUpperInvariant(model.Answer[0])}."
        },

        1 => model with
        {
            HintsUsed = 2,
            Message = $"Hint 2: the word ends with {char.
ToUpperInvariant(model.Answer[^1])}."
        },

        _ => model with
        {
            Message = "No more hints available."
        }
    };
```



This method uses pattern matching on the number of hints already used. The first hint reveals the first letter. The second hint reveals the last letter. Later hint requests leave the count unchanged and show a message.

5. Now find the existing `ShowHint` case in `Update` and replace this:

```
ShowHint when model.State is Playing =>
    model with
    {
        Message = $"Hint: the word has {model.Answer.Length} letters."
    },
```

6. with this:

```
ShowHint when model.State is Playing =>
    ShowWordHint(model),
```



The Blazor page does not need to change because it already displays `model.Message`, and the shared command bar already raises `ShowHint`.

7. Run the app and test **Word Guess**.
8. Start a game, click **Hint**, then click **Hint** again.
9. Submit a guess and confirm that hints do not reduce the number of attempts remaining.

This variation is small, but it shows an important principle. A new feature is easier to add when the existing design has a clear place for it. The hint count belongs in the model. The hint rule belongs in the rule class. The hint button already belongs in the shared command bar.

Adding difficulty levels to Number Logic

Number Logic currently has a fixed difficulty: four unique digits and eight attempts. That is a good default, but puzzle games often benefit from difficulty levels.

We will add three levels, as shown in *Table Appendix B.5*:

Difficulty	Secret length	Maximum guesses
Easy	3 digits	10 guesses
Normal	4 digits	8 guesses
Hard	5 digits	7 guesses

Table Appendix B.5: Three difficulty levels

This variation affects the number game more than the word hint change affected Word Guess. It changes the model, the rules, and the page. It should not change the shared command bar or shared action union.

A safe AI prompt would be:

```
In the PuzzleArcade Blazor project, add difficulty levels to Number Logic.

Requirements:
- Easy uses 3 unique digits and 10 guesses.
- Normal uses 4 unique digits and 8 guesses.
- Hard uses 5 unique digits and 7 guesses.
- Keep GameState and PuzzleAction unchanged.
- Keep GameCommandBar unchanged.
- Update only the NumberLogic model, rules, and page if needed.
- The player should choose difficulty before starting a game.
- Show the changed files only.
```

Implementing Number Logic difficulty levels

Now we can try implementing difficulty levels in the Number Logic game:

1. In the `Games/NumberLogic` folder, create a new file named `DifficultyLevel.cs`:

```
namespace PuzzleArcade.Games.NumberLogic;

public enum DifficultyLevel
{
    Easy,
```

```
    Normal,
    Hard
}
```



An `enum` is a good fit here. Each difficulty level is just a named option. The settings for each option will live in the rules class.

2. Update `NumberLogicModel.cs` so the model stores the current difficulty:

```
namespace PuzzleArcade.Games.NumberLogic;

using PuzzleArcade.Games.Shared;

public sealed record class NumberLogicModel(
    GameState State,
    DifficultyLevel Difficulty,
    string Secret,
    int SecretLength,
    IReadOnlyList<NumberLogicAttempt> Attempts,
    int MaxGuesses,
    string Message)
{
    public int AttemptsUsed => Attempts.Count;

    public int AttemptsRemaining =>
        Math.Max(0, MaxGuesses - AttemptsUsed);

    public bool CanGuess => State is Playing
        && AttemptsRemaining > 0;
}
```

3. In `NumberLogicRules.cs`, add a small settings record and a helper method near the top of the class:

```
private sealed record class DifficultySettings(
    int SecretLength,
    int MaxGuesses);

private static DifficultySettings GetSettings(DifficultyLevel difficulty)
=>
    difficulty switch
    {
        DifficultyLevel.Easy => new(SecretLength: 3, MaxGuesses: 10),
```

```
        DifficultyLevel.Normal => new(SecretLength: 4, MaxGuesses: 8),
        DifficultyLevel.Hard => new(SecretLength: 5, MaxGuesses: 7),
        _ => new(SecretLength: 4, MaxGuesses: 8)
    };
```

4. Update `CreateInitial` to change it into a full method that creates a difficulty level and settings and sets the difficulty:

```
public static NumberLogicModel CreateInitial()
{
    DifficultyLevel difficulty = DifficultyLevel.Normal;
    DifficultySettings settings = GetSettings(difficulty);

    return new(
        State: new NotStarted(),
        Difficulty: difficulty,
        Secret: "",
        SecretLength: settings.SecretLength,
        Attempts: [],
        MaxGuesses: settings.MaxGuesses,
        Message: "Start the game to choose a hidden digit sequence.");
}
```

5. Replace `StartNewGame` with this version:

```
private static NumberLogicModel StartNewGame(
    DifficultyLevel difficulty)
{
    DifficultySettings settings = GetSettings(difficulty);
    string secret = CreateSecret(settings.SecretLength);

    return new(
        State: new Playing(TurnsTaken: 0),
        Difficulty: difficulty,
        Secret: secret,
        SecretLength: settings.SecretLength,
        Attempts: [],
        MaxGuesses: settings.MaxGuesses,
        Message: "A hidden digit sequence has been chosen. Make your
first guess.");
}
```

6. In the `Update` method, change the `StartGame` case so it starts with the difficulty stored in the model:

```
StartGame _ => StartNewGame(model.Difficulty),
```



The rest of the rule class should already use `model.SecretLength` and `model.MaxGuesses`, so most validation and win-or-loss logic can stay the same. That is the benefit of using the model instead of hardcoded values throughout the code.

7. In `Pages/NumberLogicPage.razor`, make changes so the player can choose the difficulty before starting. Update the code block, as highlighted in the following code:



The code block is available in the online file starting at line 65: `https://github.com/markjprice/c-cpp-cs/blob/main/code/AppendixB/PuzzleArcade/Pages/NumberLogicPage.razor#L65`

```
private NumberLogicModel model = NumberLogicRules.CreateInitial();
private PuzzleAction lastAction;
private DifficultyLevel selectedDifficulty
    = DifficultyLevel.Normal;

private int SelectedSecretLength =>
    selectedDifficulty switch
    {
        DifficultyLevel.Easy => 3,
        DifficultyLevel.Normal => 4,
        DifficultyLevel.Hard => 5,
        _ => 4
    };

private string StatusDetail =>
    model.State switch
    {
        Playing => $"{model.Difficulty} mode. Attempts remaining: {model.
AttemptsRemaining}",
        Won => $"Solved in {model.AttemptsUsed} attempts.",
        Lost => "The round is complete.",
        _ => model.Message
    };

private void Dispatch(PuzzleAction action)
{
    lastAction = action;

    if (action is StartGame && model.State is not Playing)
    {
```

```
        model = model with
        {
            Difficulty = selectedDifficulty
        };
    }

    model = NumberLogicRules.Update(model, action);
}
```

8. In `Pages/NumberLogicPage.razor`, add the following markup above the `GameCommandBar`:



The markup to add is available in the online file starting at line 13: `https://github.com/markjprice/c-cpp-cs/blob/main/code/AppendixB/PuzzleArcade/Pages/NumberLogicPage.razor#L13`

```
<div class="setting-row">
  <label for="number-difficulty">Difficulty:</label>

  <select id="number-difficulty"
          @bind="selectedDifficulty"
          disabled="@(model.State is Playing)">
    <option value="@DifficultyLevel.Easy">Easy</option>
    <option value="@DifficultyLevel.Normal">Normal</option>
    <option value="@DifficultyLevel.Hard">Hard</option>
  </select>
</div>
```

9. In `Pages/NumberLogicPage.razor`, update the command bar so its maximum guess length uses the selected difficulty:

```
<GameCommandBar State="model.State"
                ShowGuessForm="true"
                GuessLabel="Digits:"
                MaxGuessLength="@SelectedSecretLength"
                GuessInputId="number-logic-input"
                OnAction="Dispatch" />
```



The page stores the selected difficulty because the shared `StartGame` action does not carry difficulty data. The page applies that selected difficulty to the model just before dispatching `StartGame`. This keeps `PuzzleAction` unchanged. That is a reasonable choice because difficulty selection is specific to Number Logic. It is not a shared arcade action used by all games.

10. Add this CSS to `wwwroot/css/app.css`:

```css
.setting-row {
  align-items: center;
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem;
  margin-bottom: 1rem;
}
```

11. Run the app and test all three levels. Easy should require three digits. Normal should require four. Hard should require five. Invalid guesses should still be rejected, and repeated digits should still be rejected.

This variation is a good example of a controlled feature. It changes the number game, but it does not disturb the word game, grid game, shared state, shared actions, or shared components.

Adding a rule change to Grid Puzzle

The Grid Puzzle currently toggles the selected tile and its orthogonal neighbors: up, down, left, and right. A simple rule variation is to include diagonal neighbors too. That changes the feel of the puzzle without changing the page or shared components.

The new rule is:

```
When the player selects a tile, toggle the selected tile and all eight
surrounding neighbors.
```

This means a middle tile in a 3 × 3 grid can affect nine tiles. A corner tile affects four tiles. An edge tile affects six tiles.

A safe AI prompt would be:

```
In the PuzzleArcade Blazor project, change the Grid Puzzle rule so selecting
a tile toggles the selected tile and all surrounding neighbors, including
diagonals.

Requirements:
- Prefer changing GridPosition.cs or GridBoard.cs.
- Do not change GridPuzzlePage.razor.
- Do not change GameState or PuzzleAction.
- Keep the game playable with the existing board.
- Show the changed file only.
```

This is the kind of change that the refactored board model makes easy:

1. In `Games/GridPuzzle/GridPosition.cs`, add a new method:

```
public IEnumerable<GridPosition> SelfAndAllNeighbors()
{
    for (int rowOffset = -1; rowOffset <= 1; rowOffset++)
    {
        for (int columnOffset = -1; columnOffset <= 1;
            columnOffset++)
        {
            yield return new GridPosition(
                Row + rowOffset,
                Column + columnOffset);
        }
    }
}
```

2. In `Games/GridPuzzle/GridBoard.cs`, find `TogglePattern`, and change it to use the new method:

```
public GridBoard TogglePattern(GridPosition position)
{
    TileState[,] copy = CopyTiles();

    foreach (GridPosition affected in
        position.SelfAndAllNeighbors())
    {
        Toggle(copy, affected);
    }

    return new GridBoard(copy);
}
```



That is the whole rule change. The page does not change. The model does not change. The shared components do not change.

3. Run the app and test **Grid Puzzle**.
4. Start a new game and click the center tile. All nine tiles should toggle.
5. Click a corner tile. Only the corner, its two orthogonal neighbors, and its diagonal neighbor should toggle because the other surrounding positions are outside the board.

This change demonstrates why board behavior was moved into `GridPosition` and `GridBoard`. Changing the puzzle's selection pattern no longer requires rewriting the Razor page.



Good practice: Change board rules where board behavior lives. A diagonal toggle rule should be implemented in `GridPosition` or `GridBoard`, not in the Razor page. The page should not need to know the details of the toggle pattern.

Reviewing the variations

You have now added three variations, each with a different scope, as shown in *Table Appendix B.6*:

Variation	Files changed	Shared components changed?
Word Guess hints	`WordGuessModel.cs`, `WordGuessRules.cs`	No
Number Logic difficulty	`DifficultyLevel.cs`, `NumberLogicModel.cs`, `NumberLogicRules.cs`, `NumberLogicPage.razor`	No
Grid Puzzle diagonal toggles	`GridPosition.cs`, `GridBoard.cs`	No

Table Appendix B.6: Three variations and the files they changed

This is the outcome we wanted. Each variation stays close to the part of the project it affects.

- The word hint variation changes word-game state and word-game rules.
- The number difficulty variation changes number-game settings and number-game UI.
- The grid rule variation changes board behavior.

None of the variations require changing the shared `GameState` lifecycle union. None require changing the shared `PuzzleAction` union. None require changing the shared command bar or status bar.

That does not mean shared code should never change. Shared code should change when the shared behavior changes. But these variations are game-specific, so they should stay in game-specific code.

What to ask AI after making the change

After each variation, you can ask an AI assistant to review the change against the project pattern:

```
Review this Puzzle Arcade change.

Check that:
- The game page still dispatches PuzzleAction values.
- The rule class still owns game-specific logic.
- The shared GameState and PuzzleAction types were not changed unnecessarily.
- The shared GameStatusBar and GameCommandBar were not changed unnecessarily.
- The change can be manually tested.

Return a concise review checklist.
```

This is a good use of AI because you are not asking it to take over. You are asking it to help you inspect the design.

You can also ask for focused test cases:

```
Generate manual test cases for the Number Logic difficulty feature.
Include Easy, Normal, and Hard modes, valid guesses, invalid guesses, repeated
digits, winning behavior, losing behavior, and reset behavior.
```

Or:

```
Generate manual test cases for the Grid Puzzle diagonal toggle rule.
Include center, edge, and corner tile selections.
```

These prompts produce review material that you can use to check the behavior yourself.

Keeping the architecture intact

These variations reinforce the main design of Puzzle Arcade:

- State belongs in models.
- Player requests are represented as actions.
- Rules produce updated models.
- The UI renders from the model.
- Shared components stay shared.
- Game-specific logic stays with the game.

This design is small enough for beginners, but strong enough to handle growth. You added hints, difficulty levels, and rule changes without turning the project into a tangle of event handlers.

That is the point of the Blazor game chapters. Modern C# is not just capable of writing game logic. It can express game logic in a way that is readable, testable, and safe to extend.

In the last section, you will test your understanding of these concepts and explore additional references and resources to deepen your knowledge and continue improving your skills.

Practice, references, and resources

In this section, you will be tested on what you've learned in this appendix and be provided with useful links to reference materials and resources.

Online material

In the separate online testing section, you will test the game logic independently of the Blazor UI, demonstrating that important rules can be verified as ordinary C# code without clicking through the browser every time:

`https://github.com/markjprice/c-cpp-cs/blob/main/docs/appendix-b-test-game-logic.md`

Check your understanding

Answer the following questions about the topics we covered in this appendix:

1. What role should AI play in this appendix: an automatic programmer or a variation partner?
2. Why is "Make the Puzzle Arcade games better" a risky prompt?
3. What should you check before asking AI to change a working project?
4. Why is it useful to ask AI for ideas before asking it for code?
5. Why should an AI prompt name target files and files that should not change?
6. What does `HintsUsed` store in `WordGuessModel`?
7. Why do Word Guess hints not consume attempts?
8. What are the three Number Logic difficulty levels added in this appendix?
9. Why is `DifficultyLevel` modeled as an enum?
10. Why does the Number Logic difficulty feature not require changing `PuzzleAction`?
11. Which Grid Puzzle types are changed to include diagonal neighbors?
12. Why should shared components stay unchanged for game-specific variations?
13. What command creates an xUnit test project?
14. Why should unit tests use known answers or secrets instead of relying on random `StartGame` behavior?
15. What does the pattern State + Action -> Rules -> New State -> UI prove about the design?



Answers are found in the GitHub repository for this book at the following link:

`https://github.com/markjprice/c-cpp-cs/blob/main/docs/Appendix-A.pdf`

Links

Testing in .NET:

`https://learn.microsoft.com/en-us/dotnet/core/testing/`

Unit testing C# in .NET using `dotnet test` and xUnit:

`https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-csharp-with-xunit`

`dotnet test`:

`https://learn.microsoft.com/en-us/dotnet/core/tools/dotnet-test`

About xUnit.net: `https://xunit.net/`

Summary

In this appendix, you learned how to:

- Use AI as a controlled variation partner rather than an automatic programmer.
- Start AI-assisted work from a known working baseline.
- Write prompts that name target files, requirements, and boundaries.
- Ask for variation ideas before asking for code.
- Review AI-generated changes against the existing project architecture.
- Use AI safely for word lists, board data, test cases, and small UI suggestions.
- Add a better Word Guess hint system with `HintsUsed`.
- Add Number Logic difficulty levels with `Easy`, `Normal`, and `Hard` modes.
- Keep game-specific difficulty behavior out of the shared `PuzzleAction` union.
- Change the Grid Puzzle toggle rule by updating board-related types.
- Keep shared components unchanged when only game-specific behavior changes.
- Create an xUnit test project for Puzzle Arcade.
- Test rule classes without launching a browser.
- Create known models for repeatable tests.
- Test Word Guess validation and winning behavior.
- Test Number Logic exact and misplaced clues.
- Test `GridBoard` and `GridPuzzleRules` behavior directly.
- Understand why UI separation makes code easier to test.

In this appendix, you used AI as a controlled variation partner rather than an automatic programmer. You added hints, difficulty levels, and a rule change while keeping shared code shared and game-specific code with the game that owns it. You also tested game logic separately from the Blazor UI, proving that the important rules can be verified as ordinary C# code.

Puzzle Arcade showed that modern C# is well suited to interactive applications. Union types, records, pattern matching, and clear models helped describe states, actions, results, guesses, clues, boards, and tile behavior. Blazor then rendered the UI from those models and turned player interactions into actions.

In *Chapter 18, Hand-Crafting a 3D Game in Unity*, you move from browser-based puzzles to a professional game engine. You use Unity to build a small 3D game by hand, learning the editor, scenes, GameObjects, components, materials, physics, and scripts. It would be impossible to master Unity in one chapter. The idea is to give you a first taste of how C# is used inside a real game development environment.

Get this book's PDF version and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.